

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)**

Кафедра компьютерных систем в управлении и проектировании (КСУП)

К ЗАЩИТЕ ДОПУСТИТЬ

Заведующий кафедрой КСУП

д-р техн. наук, проф.

_____ Ю.А. Шурыгин

«_____» _____ 2025 г.

Бакалаврская работа

по направлению подготовки

09.03.01 – Информатика и вычислительная техника

**РАЗРАБОТКА ЛОГИКИ ДОСТИЖЕНИЙ ПОЛЬЗОВАТЕЛЯ В
КРОССПЛАТФОРМЕННОМ ПРИЛОЖЕНИИ ФОРМИРОВАНИЯ
ПОЛЕЗНЫХ ПРИВЫЧЕК «NAVITUALIZE»**

Студент гр. 571-1

_____ И. В. Круглов

«___» _____ 2025 г.

Руководитель

старший преподаватель каф. КСУП

_____ Е. С. Мурзин

«___» _____ 2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)**

Кафедра компьютерных систем в управлении и проектировании (КСУП)

УТВЕРЖДАЮ

Заведующий кафедрой КСУП

д-р техн. наук, проф.

_____ Ю. А. Шурыгин

«___» _____ 20 __г.

**Индивидуальное задание
на выполнение выпускной квалификационной работы**

Студенту Круглову Илье Владимировичу гр. 571-1 факультета вычислительных систем (ФВС)

1. Тема работы: Разработка логики достижений пользователя в кроссплатформенном приложении формирования полезных привычек «Habitualize» (утверждена приказом от _____ № _____).

2. Цель работы: – создать программный комплекс, который поможет пользователю эффективно использовать свое время и предоставит возможность планировать свои обязанности.

3. Содержание работы (перечень вопросов, подлежащих разработке):

3.1 разработка интерфейса и логики для создания привычек;

3.2 создание календаря для отслеживания привычек по дням;

3.3 разработка системы достижений и отслеживания прогресса достижений;

3.4 создание интерфейса для просмотра статистики по созданным привычкам;

3.5 настройка подключения к облачной базе данных Firebase для сохранения и загрузки пользовательских данных.

4. Дата выдачи задания: «26» мая 2025 г.

5. Дата сдачи работы на кафедру: «31» июня 2025 г.

Руководитель от университета

Мурзин Е.С., старший преподаватель каф. КСУП «__» ____ 2025 г.

ЗАДАНИЕ СОГЛАСОВАНО:

Консультант по нормам и требованиям ЕСКД

Черкашин М.В., доцент каф КСУП, к.т.н. «__» ____ 2025 г.

Студент

группы 571-1

Круглов И.В. «26» мая 2025 г.

Реферат

Бакалаврская работа 43 страницы, 30 рисунков, 0 таблиц, 14 источников, 0 приложений.

РАЗРАБОТКА, ПРОГРАММИРОВАНИЕ, ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ, C#, ГЕЙМИФИКАЦИЯ, КРОССПЛАТФОРМЕННОЕ ПРИЛОЖЕНИЕ

Объектом разработки является кроссплатформенное приложение для формирования полезных привычек.

Цель работы – создать программный комплекс, который поможет пользователю эффективно использовать свое время и предоставит возможность планировать свои обязанности. Приложение обязано работать как на мобильных устройствах, так и на персональных компьютерах.

Для разработки использовалась Visual Studio, вместе с пакетами NuGet. Была внедрена облачная база данных Firebase. Языком разработки является .NET Multi-platform App UI (MAUI).

Реализация программного комплекса проходила в несколько этапов: создание базовых функциональных возможностей, внедрение облачной базы данных и реализация геймификации в приложении.

В приложении пользователь может составлять полезные привычки, отслеживать свой прогресс, и с помощью использования принципов геймификации у пользователя будет мотивация использовать программный комплекс, а также само приложение не будет восприниматься как сложное программное обеспечение.

Abstract

Bachelor's thesis 43 pages, 30 illustrations, 0 tables, 11 sources, 14 appendixes.

DEVELOPMENT, PROGRAMMING, SOFTWARE, C#,
GAMIFICATION, CROSSPLATFORM APPLICATION

The object of development is cross platform software that aims to form useful habits in user's daily life.

The goal of this thesis is to create software package, that helps the user to use time more efficiently and gives them an opportunity to plan how to tackle daily responsibilities. Software must run properly on both mobile devices as well as personal computers.

Visual Studio and NuGet packages were used during the development. Additionally, Firebase was used as cloud storage for user's data. As programming language .NET Multi-platform App UI (MAUI) was used.

The development cycle was divided in few stages: developing basic functions, linking Firebase cloud storage to software and applying the principles of gamification to encourage user to interact with software.

In the application itself users can develop useful habits and track their progress in chosen tasks throughout the day. This software was designed with an idea of gamifying chores and tasks in mind, which will allow users to perceive their set goals as more rewarding.

Оглавление

Введение	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1 ПОСТАНОВКА ПРОБЛЕМЫ	7
1.2 ОБЗОР АНАЛОГОВ	9
1.3 АКТУАЛЬНОСТЬ РАБОТЫ	13
1.4 ТРЕБОВАНИЯ К ПРОГРАММНОМУ КОМПЛЕКСУ	14
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА	16
2.1 ВЫБОР СТЕКА ТЕХНОЛОГИЙ	16
2.2 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА	21
2.3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ ХРАНЕНИЯ ДАННЫХ	23
2.4 ПРОЕКТИРОВАНИЕ СИСТЕМЫ ПРИВЫЧЕК И ДНЕВНИКА НАСТРОЕНИЯ	27
3 ТЕСТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА	30
3.1 СИСТЕМА СОЗДАНИЯ И РЕДАКТИРОВАНИЯ ПРИВЫЧЕК	30
3.2 КАЛЕНДАРЬ И ДНЕВНИК НАСТРОЕНИЯ	33
3.3 ОТСЛЕЖИВАНИЕ ПРОГРЕССА И СИСТЕМА ДОСТИЖЕНИЙ	36
Заключение	39
Список использованных источников	41

Диск CD-R (Накопитель 4 Gb)

В конверте на обороте
обложки

Пояснительная записка к ВКР

43 стр.

Презентация дипломного проекта

16 слайд.

Отчет о проверке на плагиат

2 стр.

Твердая копия презентации

6 экз.

Введение

Цель работы – разработка кроссплатформенного программного комплекса для формирования полезных привычек с помощью геймификации.

Область исследования основывается на внедрении и использовании современных технологий и программного обеспечения для создания приложения, которое поможет повысить эффективность работы пользователя, а также мотивировать на улучшение своей работоспособности.

Область применения программного комплекса включает в себя формирование полезных привычек, улучшение эффективности работы, а также повышение мотивации для выполнения задач.

Для разработки программного комплекса необходимо реализовать:

- 1) интерфейс и логику для создания и редактирования привычек;
- 2) календарь для отслеживания привычек по дням;
- 3) систему достижений и отслеживания прогресса достижений;
- 4) интерфейс для просмотра статистики по созданным привычкам;
- 5) подключение к облачной базе данных Firebase для сохранения и загрузки пользовательских данных.

Также программный комплекс использует различные пакеты NuGet для упрощения реализации и развертывания приложения на нескольких платформах.

В результате работы созданное приложение оказывает положительное влияние на поведенческие шаблоны пользователя, что позволяет повысить личную эффективность при выполнении различных задач.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Постановка проблемы

В современном мире для каждой работы необходим человеческий труд, на количество выполненной работы влияет работоспособность. Работоспособность – крайне важная характеристика для любого человека. На работоспособность крайне часто влияет среда, в которой находится человек, а также сильное влияние оказывает способность планировать свои задачи. На рисунке 1.1 изображен график работоспособности человека (в условных единицах).

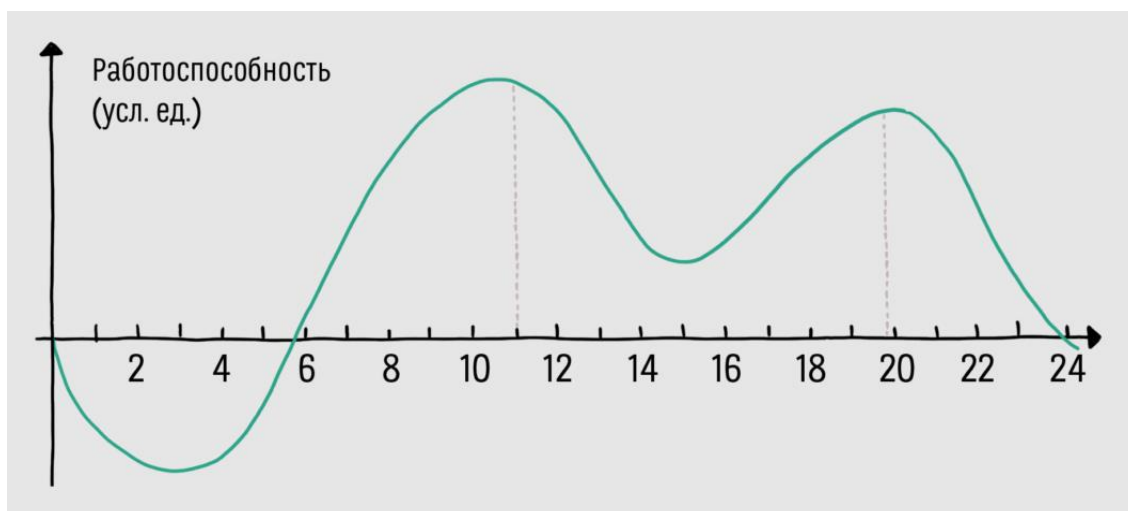


Рисунок 1.1 – График работоспособности человека

График создан для среднестатистического человека, который работает с 9 утра до 17 вечера. В ночное время (с 0 часов до 6 утра) график находится в отрицательных значениях, потому что, человек находится в фазе сна. Данный график не учитывает степень планирования задач, а также людей, которые работают в ночную смену, так как для них данные будут выглядеть иначе. На графике видно два пика работоспособности: с 10 до 12 часов утра и с 17 до 20 часов вечера. Эти временные отрезки являются самым продуктивным временем для человека. Для утреннего отрезка, очевидно

почему он является пиком, люди пробуждаются, и после восстановления сил начинают рабочий день. Затем начинается работа и человек получает пик своей работоспособности. Затем после 12 часов начинается сильное снижение работоспособности и продолжается до конца рабочего дня. В этот отрезок начинается утомление индивида, организм мало фокусируется на работе, а сам человек готовится к завершению рабочего дня. В вечернее время происходит скачок работоспособности, люди хотят выполнить задачи и отправиться домой быстрее. Дома начинается смена деятельности и человек получает второе дыхание для выполнения повседневных задач.

Для решения проблемы низкой работоспособности человек может сменить обстановку вокруг себя, что зачастую сложно выполнить, или начать планирование задач. Второй пункт проще выполнить, и может помочь для большинства людей, но имеет серьезный минус – планирование это навык, который необходимо развивать для эффективного использования.

Людям сложно продолжать выполнение необязательных задач без существенного стимула, как раз для решения проблемы мотивации проводится разработка программного комплекса «Habitualize».

Также были использованы принципы геймификации для повышения вовлеченности при использовании программы. Функциональные возможности приложения просты для понимания, пользователь получает обратную связь от программы в виде достижений или повышения уровня, что позволяет пользователю отслеживать свой прогресс и видеть результат.

Если пользователю преподнести приложение как игру, то восприятие приложения изменится, из приложения для работы или повседневных дел, на игру, в которой человек составляет свои новые привычки и планы для повышения работоспособности.

В результате приложение можно будет использовать как для повседневных дел, так и для работы, что значительно повысит эффективность индивида. Однако программный комплекс можно использовать для достижения своих личных целей.

Для людей также важны социальные функции, которые есть в приложении, с помощью этого люди смогут как соревноваться друг с другом, так и поддерживать в формировании полезных привычек.

1.2 Обзор аналогов

Существуют различные приложения для формирования полезных привычек и планирования, подробно рассмотрим некоторые из них.

SingularityApp [1]. На рисунке 1.2 изображено несколько привычек, созданных в SingularityApp, в календаре приложения отображается отрезок, обозначающий сколько дней подряд была выполнена привычка. Привычки не имеют каких-либо дополнительных условий для выполнения, пользователь просто вводит выполнена привычка или нет, но можно отметить выполнена привычка полностью или лишь на половину. Также присутствует небольшая статистика в приложении для лучшей привычки (которая выполнялась больше всего раз) и для среднего выполнения привычек (количество выполненных привычек делится на количество всех дней использования приложения). В приложении присутствует статистика для всех привычек отдельно, указывается процент выполнения привычки и самая длинная серия выполненных привычек подряд.



Рисунок 1.2 – Основной интерфейс SingularityApp

Данное приложение доступно как на мобильных устройствах, так и на персональных компьютерах (присутствует и веб-версия приложения). Однако присутствует существенный минус – основные функциональные возможности платные. В бесплатной версии можно создать и отслеживать только 3 привычки.

Tick It [2]. Это приложение позиционирует себя не только как трекер привычек, но и как программа для достижения целей.

Присутствует большое количество настроек для создания привычки (привычку можно сделать регулярной, вредной или одноразовой), отслеживание выполнения и присутствует возможность сделать выходной для привычки, то есть пропущенный день не будет учитываться в статистике.

Для созданной привычки можно выбрать каким образом она будет учитываться: либо количество повторений, либо продолжительность выполнения. На рисунке 1.3 указан экран создания привычки.

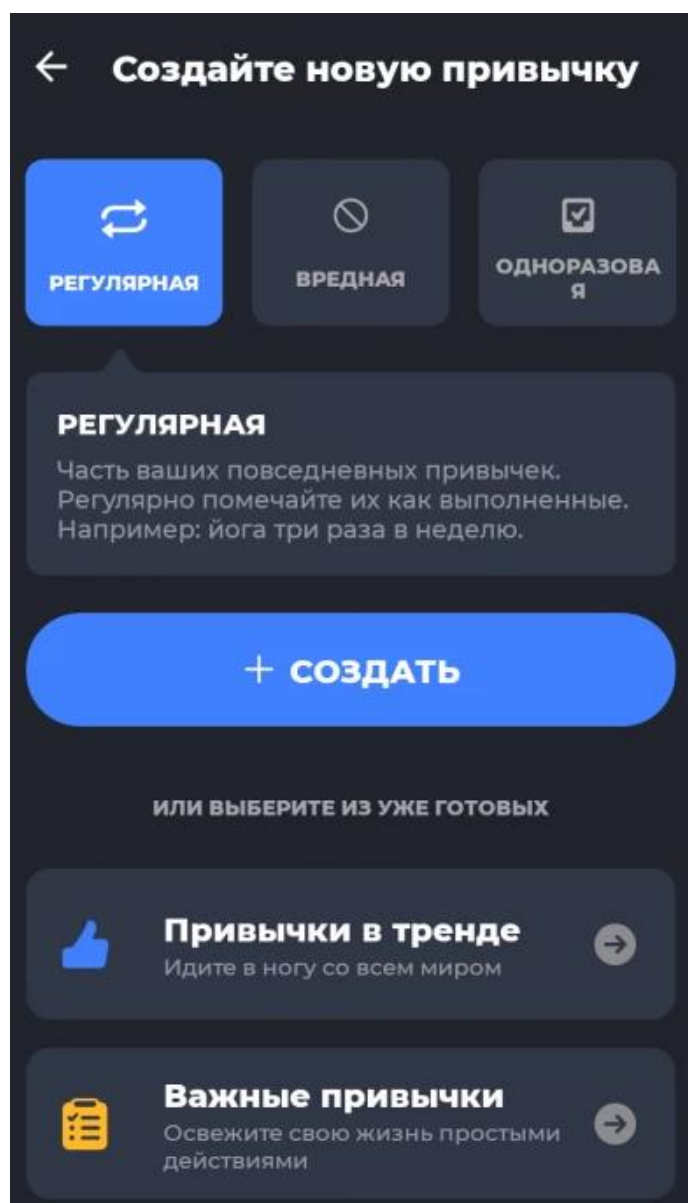


Рисунок 1.3 – Интерфейс создания привычки в Tick It

Данное приложение присутствует только на мобильных устройствах, что является одним из недостатков программы, но все функциональные возможности доступны в бесплатной версии.

Everyday [3]. Приложение, созданное только для отслеживания выполнения привычек. Эта программа обладает простым и понятным интерфейсом, на главной странице сразу видно календарь в котором указано пропущена привычка или выполнена (рисунок 1.4).

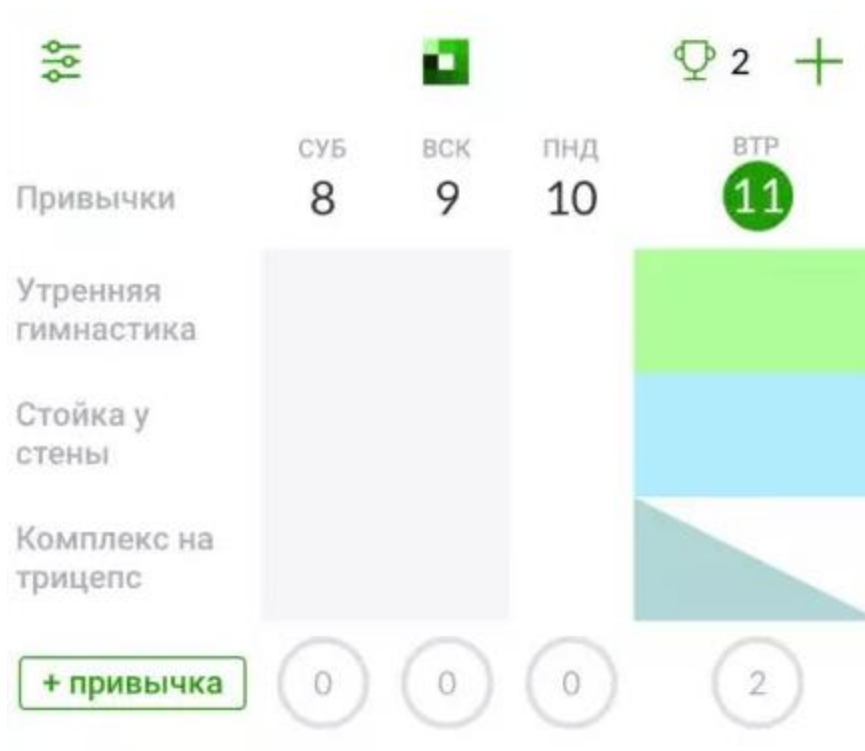


Рисунок 1.4 – Главный экран приложения

Выполненная привычка отмечается квадратом, а пропущенная треугольником, так пользователь может продолжить серию из привычек, несмотря на то что один или несколько дней пропущены. Это способствует продолжению выполнения привычки. Присутствует статистика (рисунок 1.5), она отображает в какие дни выполнены привычки, но интерфейс статистики не совсем прост для понимания.

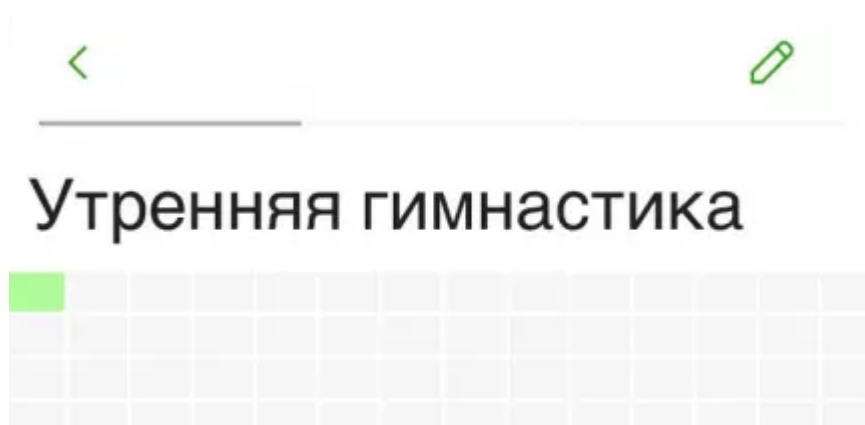


Рисунок 1.5 – Окно статистики

Это приложение, как и Tick It доступно только на мобильных

устройствах и не обладает веб-версией. Также как и в SingularityApp можно загрузить только 3 привычки в бесплатной версии.

Обзор аналогов позволил нам посмотреть на главные достоинства и недостатки приложений. Из всего выше перечисленного можно отметить:

- 1) приложение должно быть кроссплатформенным, для увеличения
- 2) количества пользователей;
- 3) в приложении основные функциональные возможности должны быть доступны без дополнительных денежных затрат;
- 4) добавление статистики позволит пользователю видеть результат
- 5) своего труда.

Также в разрабатываемое приложение можно добавить дополнительные возможности для поддержания высокой вовлеченности при использовании приложения (использовать принципы геймификации).

1.3 Актуальность работы

Тема данной работы важна, ведь в последнее десятилетие статистика по снижению работоспособности продолжает увеличиваться. Очевидно, что присутствует корреляция с профессиональным выгоранием, поэтому обратимся к научной статье от Forbes.

«...43% работающих россиян хоть раз за свою карьеру испытывали профессиональное выгорание — пятая часть сотрудников все еще пребывает в этом состоянии». Также считается что около 70% работников считают, что их работа бесполезна, а 15% испытывают это каждый рабочий день.

Основными причинами этого состояния являются: стресс на работе, отсутствие признания значимости от руководства, несоответствие личных ценностей ценностям компании.

На вышеперечисленные факторы крайне сложно повлиять, но мы можем повлиять на снижение стресса на работе. Основная причина стресса — не выполнение поставленных задач. Это может быть связано с ошибками в планировании своего рабочего дня: расплывчатая формулировка задания,

отсутствие конкретных метрик для отслеживания прогресса выполнения задания, отсутствие приоритетов, недостаток контроля.

Для решения этих проблем проводится разработка приложения «Habitualize». Люди крайне часто пользуются мобильными устройствами, но доступ к ним может быть ограничен, поэтому одним из главных преимуществ программного комплекса должна быть кроссплатформенность.

1.4 Требования к программному комплексу

Для любой разрабатываемой системе должен быть список требований, который необходимо выполнить.

Основные требования к программному комплексу:

1) разработать адаптивный интерфейс – так как приложение будет кроссплатформенным адаптивный интерфейс крайне важен. На мобильном устройстве и на персональном компьютере приложение обязано выглядеть одинаково, не зависимо от размеров экрана;

2) создать логику для создания привычек – необходимо создать логику для нескольких категорий привычек, создание, удаление и редактирование. Всего будет четыре категории привычек для: растений, тренировок, чтения, личных привычек;

3) реализовать календарь внутри приложения – календарь с событиями будет крайне полезен для пользователя. Это откроет возможность более гибкого планирования задач;

4) разработать систему достижений – система достижений является одной из систем геймификации приложения. Пользователь будет мотивирован наградой для взаимодействия со всеми разделами приложения и с продолжением использования всех систем программного комплекса;

5) разработать систему прогресса – система прогрессии — это развитие геймификации, даже простое увеличение уровня позволит пользователю получить мотивацию, и как следствие, увеличить

вовлеченность при использовании приложения;

6) создать ежедневные задания, связанные с прогрессом в приложении – ежедневные задания будут заставлять пользователя заходить в приложение каждый день, что не только увеличит личную эффективность пользователя в выполнении своих задач и привычек, но и мотивирует его продолжать использовать приложение;

7) создать систему сохранения пользовательских данных в облачной базе данных.

Необходимость облачного хранения данных является следствием кроссплатформенности программного комплекса. Если пользователь не будет иметь доступ к устройству, с которого он обычно заходит в приложение, то все данные на другом (новом) устройстве должны быть синхронизированы и сохранены.

Выполнив все эти требования, можно считать разработку программного комплекса успешно выполненной.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА

2.1 Выбор стека технологий

Необходимо выбрать стек технологий, который будет удовлетворять требованиям описанными в пункте 1.4. Для языка программирования необходимо выбрать такой, с помощью которого можно легко развернуть систему на нескольких платформах сразу.

Фреймворк .NET Multi-platform App UI (далее в работе будет использоваться сокращение MAUI) отвечает требованию кроссплатформенности. В этом фреймворке используется язык C# для кода и XAML для разметки. Приложения с этим фреймворком можно разработать для Windows, macOS, iOS и Android, используя один и тот же код.

Преимущества MAUI по сравнению с другими фреймворками:

- 1) в процессе разработки создается единый проект, который использует один код для всех платформ;
- 2) предоставляется доступ к нативным API для каждой платформы, в том числе к аппаратным возможностям платформ;
- 3) при создании приложений используется язык C#, который является довольно производительным, что повышает быстродействие программы;
- 4) присутствует поддержка привязки данных;
- 5) гибкая возможность настроек поведения интерфейса;
- 6) присутствует горячая перезагрузка, что слегка упрощает разработку приложения.

MAUI объединяет API всех платформ в один единый, который как раз и поддерживает код для всех операционных систем, также предоставляется возможность при необходимости добавить для отдельной платформы специфические для неё функциональные возможности.

На рисунке 2.1 изображена схема работы MAUI.

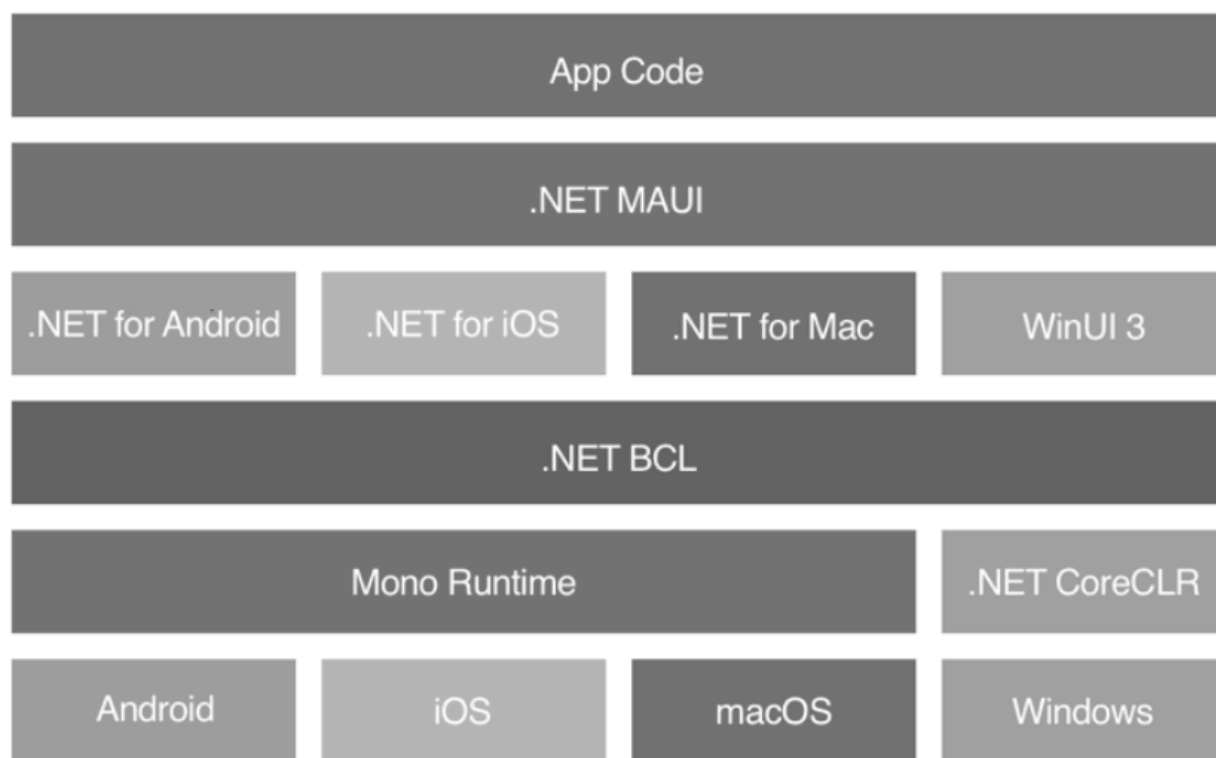


Рисунок 2.1 – Схематически представленная работа MAUI

Фреймворк опирается на ряд субплатформ, через которые идет взаимодействие с каждой отдельной операционной системой. В общем случае код приложения сначала обращается к .NET MAUI, а затем обращается к субплатформе конкретной операционной системы. Фреймворк также позволяет коду обращаться напрямую к субплатформам.

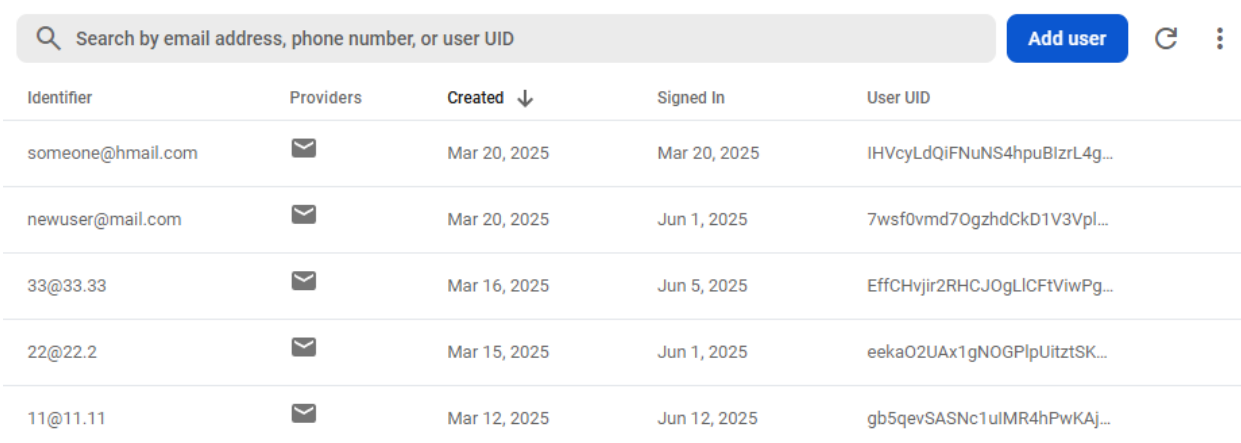
Все субплатформы работают поверх библиотеки классов .NET Base Class Library (BCL), эта библиотека позволяет абстрагироваться от деталей реализации конкретной платформы, все зависит от среды, в которой выполняется код. Для Android, iOS, macOS среду выполнения для приложения обеспечивает встроенный фреймворк Mono. На платформе Windows среда выполнения предоставляется .NET CoreCLR.

Разработчик с помощью MAUI может определить общую единую логику для приложения на всех платформах, а при необходимости можно использовать специфические для каждой платформы функциональные возможности.

Для выполнения еще одного требования к разработке нужно подключить облачную базу данных. Необходима именно облачная БД, так как прогресс и данные приложения должны сохраняться между платформами. Была выбрана платформа Firebase.

Firebase – набор инструментов и сервисов для разработки мобильных и веб-приложений. С помощью этого инструментария можно развернуть серверную логику, подключить базу данных и настроить авторизацию пользователей.

Firebase Authentication – позволяет обеспечить безопасный и простой способ регистрации и входа пользователей в приложение. Вход и регистрация проходит с помощью электронной почты и пароля, либо с помощью социальных сетей. На рисунке 2.2 изображена таблица, в которой хранятся данные для авторизации пользователей.



The screenshot shows the Firebase Authentication console interface. At the top, there is a search bar with the placeholder text "Search by email address, phone number, or user UID" and a blue "Add user" button. Below the search bar is a table with the following columns: Identifier, Providers, Created, Signed In, and User UID. The table contains five rows of user data.

Identifier	Providers	Created ↓	Signed In	User UID
someone@hmail.com		Mar 20, 2025	Mar 20, 2025	IHVcyLdQiFNuNS4hpuBlzrL4g...
newuser@mail.com		Mar 20, 2025	Jun 1, 2025	7wsf0vmd70gzhdCkD1V3Vpl...
33@33.33		Mar 16, 2025	Jun 5, 2025	EffCHvjir2RHCJOgLICFtVivPg...
22@22.2		Mar 15, 2025	Jun 1, 2025	eekaO2UAx1gNOGPipUitztSK...
11@11.11		Mar 12, 2025	Jun 12, 2025	gb5qevSASnc1ulMR4hPwKAj...

Рисунок 2.2 – Таблица для хранения данных авторизации

Здесь первичным ключом является уникальный идентификатор пользователя, на данной таблице методом входа является вход через электронную почту. Пароль также хранится здесь, но он зашифрован и не указан в таблице.

Firebase Realtime Database – облачная база данных, основанная на NoSQL, которая позволяет хранить и синхронизировать данные в реальном времени, является оптимизированной для автономного использования.

На рисунке 2.3 изображено как данные хранятся в базе данных, а на рисунке 2.4 указано в каком виде хранятся отдельная привычка.



Рисунок 2.3 – Хранение всех данных в базе

Данные хранятся в виде дерева, корнем которого является ссылка на базу данных (для подключения в коде программы). После указания ссылки указывается что хранится в ветви, затем само значение. Рассмотрим, как выглядят отдельно взятые данные.

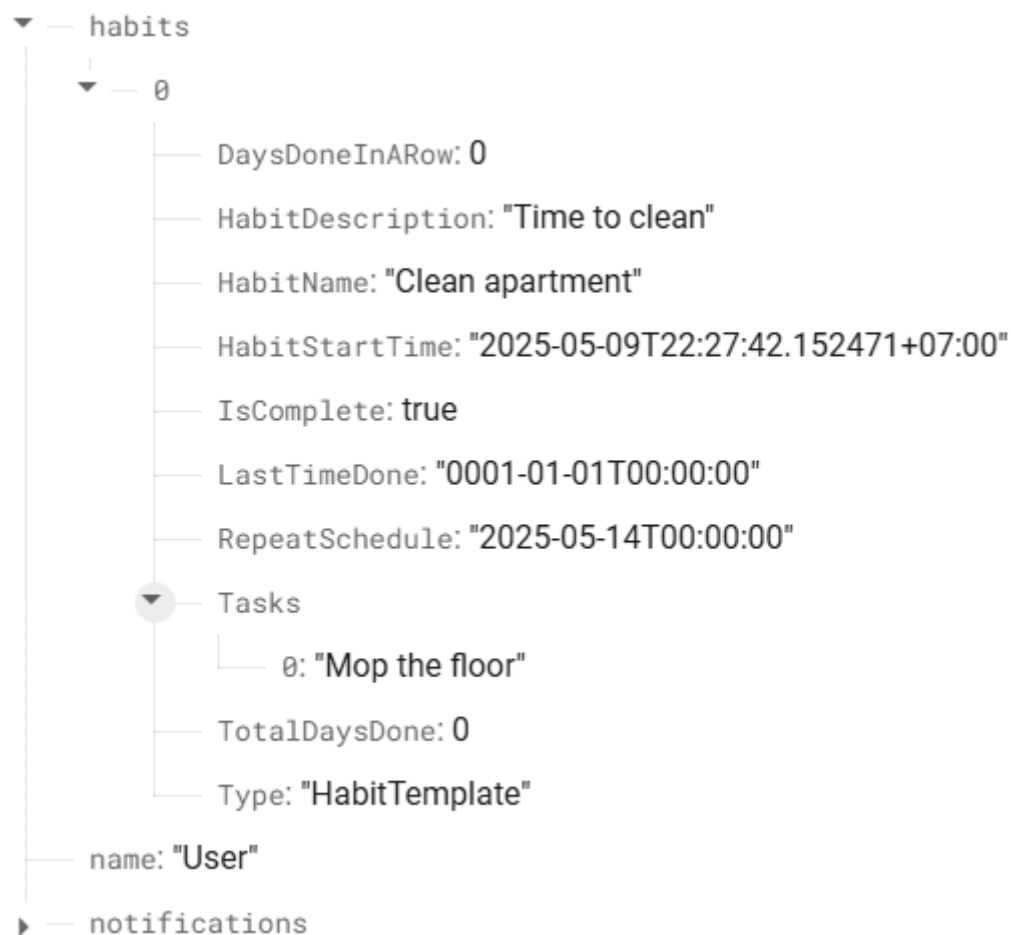


Рисунок 2.4 – Хранение экземпляра класса

Сами данные хранятся в формате json, на рисунке видно что привычки и задачи хранятся в виде списков. Хранение в формате json обязывает к сериализации и десериализации данных, но позволяет хранить большие массивы в компактных и понятных для разработчика файлах.

Все данные в Firebase уже автоматически защищены от стороннего вмешательства, также одним из основных преимуществ является масштабируемость системы.

Для разработки программного комплекса использовались пакеты NuGet, которые доступны в Visual Studio. Рассмотрим каждый пакет по отдельности:

1. «CommunityToolkit.Maui»

Инструментарий для более гибкой работы с MAUI, содержит коллекцию шаблонов для: анимаций, поведения интерфейса, конвертеров и

работы с различными платформами.

2. «CommunityToolkit.Mvvm»

Пакет, который добавляет несколько библиотек для реализации шаблона программирования Model-View-Model-View.

3. «FirebaseAuthentication.net»

Библиотека, необходимая для работы с аутентификацией и авторизацией пользователей в Firebase.

4. «FirebaseDatabase.net»

Библиотека, необходимая для сохранения, загрузки и синхронизации с облачной базой данных Firebase.

5. «Microsoft.AspNetCore.SignalR.Client»

Пакет, необходимый для работы с системой чатов внутри приложения.

6. «Plugin.Maui.Calendar»

Плагин, который добавляет календарь для интерфейса, в котором можно создавать и редактировать события, которые отмечаются в календаре.

7. «Syncfusion.Maui.Charts»

Библиотека, используемая для построения графов и диаграмм в интерфейсе MAUI.

Все эти пакеты в своей совокупности позволяют более гибко настроить приложение и добавить несколько полезных функций.

2.2 Архитектура программного комплекса

В программном комплексе реализован архитектурный шаблон Model-View-Model-View (MVVM). Этот шаблон модификация Presentation Model, и состоит из трех компонентов: модели, модели представления и представления.

Модель описывает используемые в приложении данные. Модели содержат логику, непосредственно связанную этими данными. В то же время модель не должна содержать никакой логики, связанной с отображением

данных и взаимодействием с визуальными элементами управления.

Нередко здесь реализуются интерфейсы `INotifyPropertyChanged` или `INotifyCollectionChanged`, которые позволяют уведомлять систему об изменениях свойств модели. Благодаря этому облегчается привязка к представлению, хотя опять же прямое взаимодействие между моделью и представлением отсутствует.

Представление определяет визуальный интерфейс, через который пользователь взаимодействует с приложением. В разрабатываемом программном комплексе представление – это код в `html`, который определяет интерфейс в виде кнопок, текстовых полей и прочих визуальных элементов. Само представление не содержит логики, вся основная логика приложения выносится в компонент `ViewModel`. Также представление не обрабатывает события, а выполняет действия в основном посредством команд.

Модель представления связывает модель и представление через механизм привязки данных. Если в модели изменяются значения свойств, при реализации моделью интерфейса `INotifyPropertyChanged` автоматически идет изменение отображаемых данных в представлении, хотя напрямую модель и представление не связаны. `ViewModel` также содержит логику по получению данных из модели, которые потом передаются в представление. И также определяет логику по обновлению данных в модели.

Итогом применения паттерна `MVVM` является функциональное разделение приложения на три компонента, которые проще разрабатывать и тестировать, а также в дальнейшем модифицировать и поддерживать.

2.3 Проектирование системы хранения данных

На рисунке 2.5 представлена диаграмм классов для привычек.

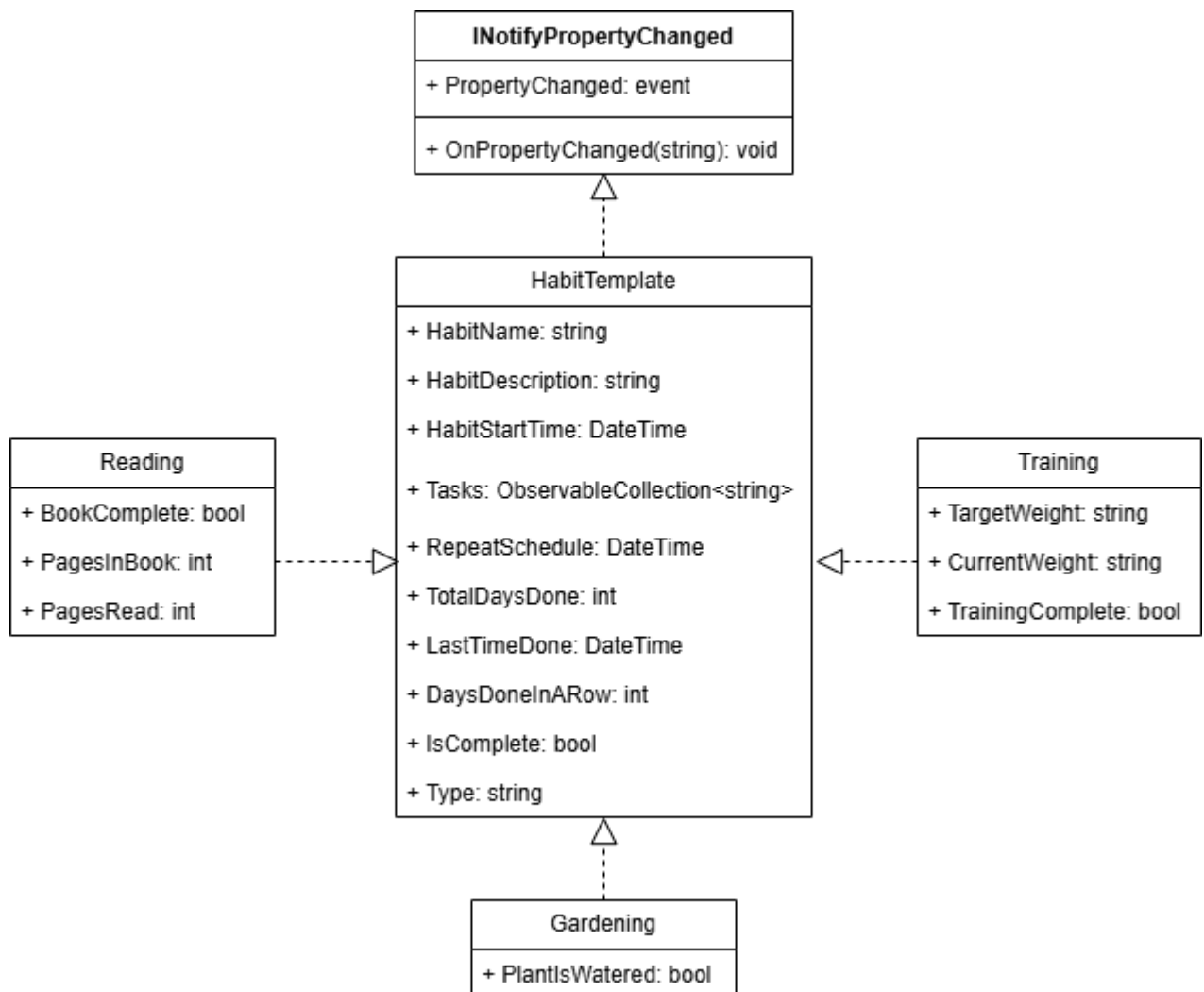


Рисунок 2.5 – Диаграмма классов привычек

Основной класс привычек – **HabitTemplate**, он является основным для всех остальных. Этот класс наследует интерфейс **INotifyPropertyChanged**, который позволяет после обновления данных во View уведомлять Model о изменениях. Далее классы остальных привычек наследуют все поля от **HabitTemplate** и добавляют свои собственные.

Основные поля класса **HabitTemplate**: название привычки, описание привычки, дата начала привычки, цикл повторения, сколько всего дней выполнено и выполнена привычка или нет.

Поле Type необходимо для сохранения в базу данных, несмотря на то что данные хранятся в формате json нельзя включить настройку, которая позволяет сохранить полиморфизм, это поле, которое заполняется автоматически, необходимо для корректного сохранения элементов в базу данных.

Одной из важных функций является сохранение и загрузка данных, на рисунке 2.6 изображен код, выполняемый при сохранении данных в Firebase.

```
public async Task SaveInFirebase(List<HabitTemplate> habits, List<AchievementsTemplate> achievements)
{
    var firebase =
        new FirebaseClient("https://habitualize-249ef-default-rtdb.europe-west1.firebaseio.com/");
    var uid = _authClient.User.Uid;

    if (uid != null)
    {
        await firebase
            .Child("user")
            .Child(uid)
            .Child("habits")
            .PutAsync(habits);
        await firebase
            .Child("user")
            .Child(uid)
            .Child("achievements")
            .PutAsync(achievements);
    }
}
```

Рисунок 2.6 – Сохранение привычек

Сначала происходит подключение к облачной базе данных по ссылке, потом записывается уникальный идентификатор пользователя в переменную. Если идентификатор не пустой, то происходит запись всех привычек и достижений, они сохраняются в виде списка, а первичным ключом является идентификатор пользователя.

Функция загрузки устроена немного сложнее, чем сохранение (рисунок 2.7).

```

public async Task<List<HabitTemplate>> LoadHabitsFromFirebase()
{
    var firebase =
        new FirebaseClient("https://habitualize-249ef-default-rtdb.europe-west1.firebaseio.com/");
    var uid = _authClient.User.Uid;
    if (uid != null)
    {
        var customHabits = await firebase
            .Child("user")
            .Child(uid)
            .Child("habits")
            .OnceAsListAsync<HabitTemplate>();
        customHabits = customHabits.Where(habit => habit.Object.Type == "HabitTemplate").ToList();
        var gardeningHabits = await firebase
            .Child("user")
            .Child(uid)
            .Child("habits")
            .OnceAsListAsync<Gardening>();
        gardeningHabits = gardeningHabits.Where(habit => habit.Object.Type == "Gardening").ToList();
        var readingHabits = await firebase
            .Child("user")
            .Child(uid)
            .Child("habits")
            .OnceAsListAsync<Reading>();
        readingHabits = readingHabits.Where(habit => habit.Object.Type == "Reading").ToList();
        var trainingHabits = await firebase
            .Child("user")
            .Child(uid)
            .Child("habits")
            .OnceAsListAsync<Training>();
        trainingHabits = trainingHabits.Where(habit => habit.Object.Type == "Training").ToList();
        List<HabitTemplate> allHabits = new List<HabitTemplate>();
        allHabits.AddRange(customHabits.Select(habit => habit.Object));
    }
}

```

Рисунок 2.7 – Функция загрузки привычек

Как и в сохранении сначала устанавливается подключение к Firebase, затем записывается уникальный идентификатор пользователя. Далее из-за ограничений Firebase для каждой привычки из всех четырех классов отдельно загружается список, который записывается в общий список всех привычек, который функция как раз возвращает.

Рассмотрим алгоритм разблокирования достижений (рисунок 2.8).

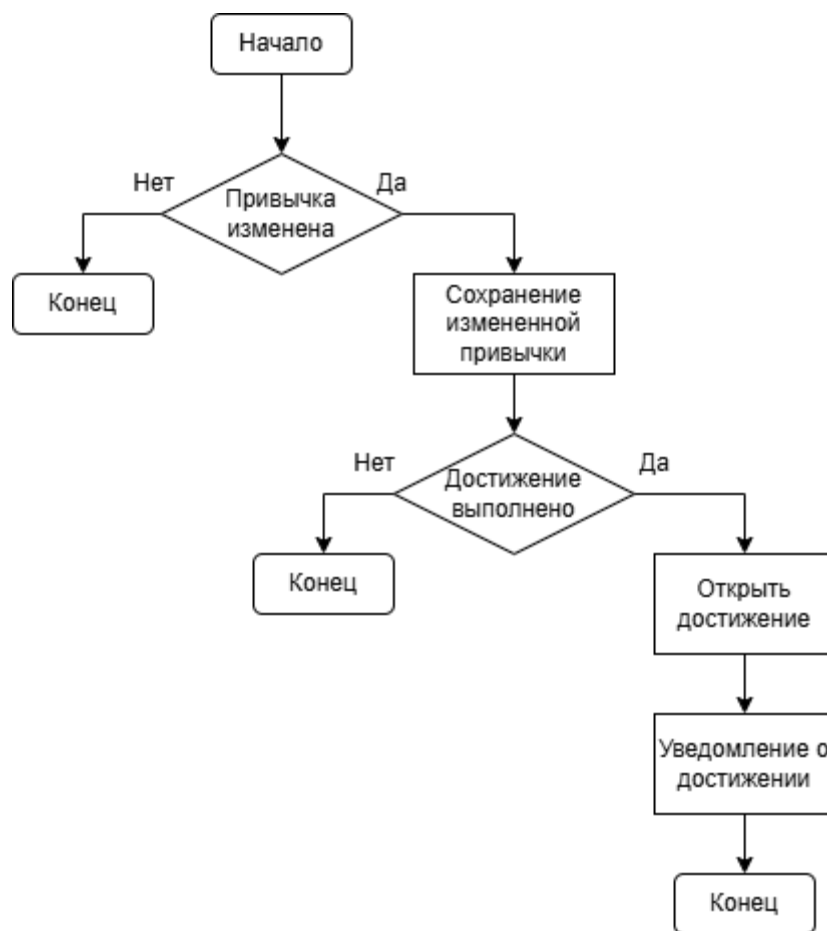


Рисунок 2.8 – Алгоритм открытия достижений

Весь алгоритм происходит на странице изменения привычки, сначала проверяется присутствуют ли изменения в привычке, если нет, то выход на предыдущую страницу. Если привычка изменена, то начинается проверка выполнения условия разблокировки достижения, если всё верно и достижение открыто, то пользователь получает уведомление о разблокированном достижении и в профиле отмечается выполненным. Если достижение не выполнено, то привычка сохраняется и происходит выход на предыдущее окно.

2.4 Проектирование системы привычек и дневника настроения

Рассмотрим сценарии использования приложения, для этого создана use-case диаграмма (рисунок 2.9).

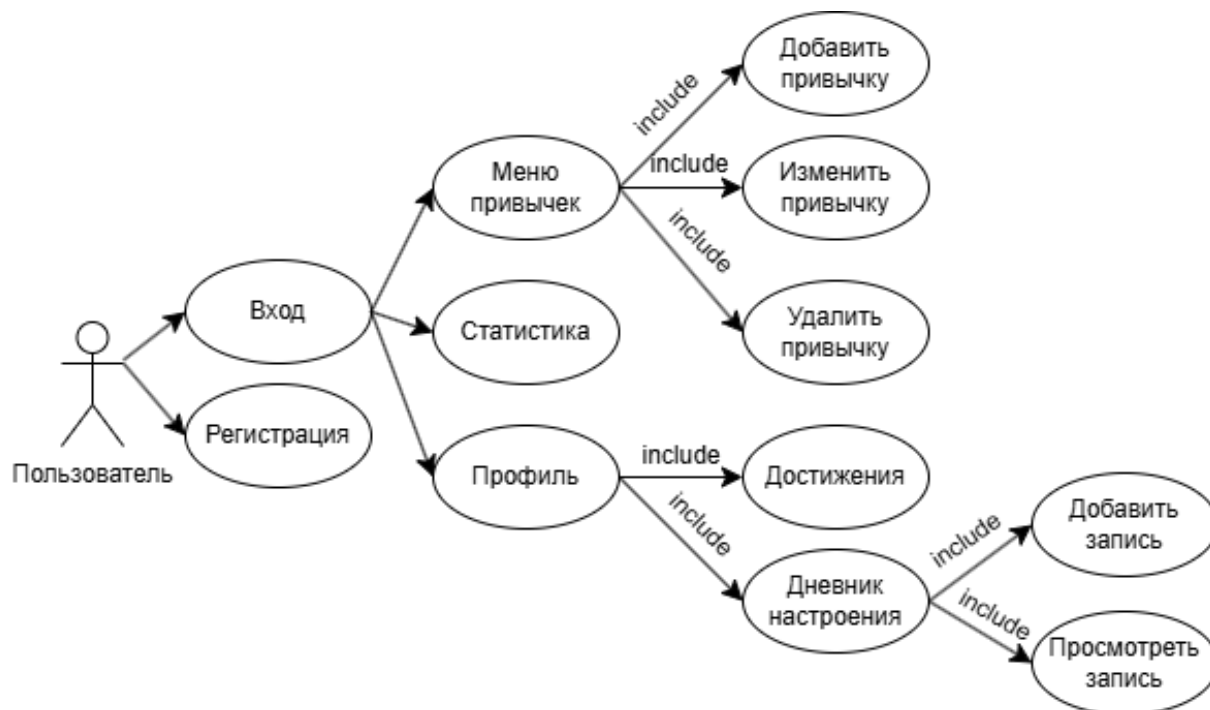


Рисунок 2.9 – Диаграмма сценариев использования

В меню привычек пользователь выбирает какой тип привычки выбрать (растения, спорт, книги или свои привычки). На этой же странице указывается статистика по всем привычкам. В профиле пользователь может просмотреть список достижений, а также добавить или просмотреть записи в дневнике настроения.

Подробнее рассмотрим меню для привычек, связанных с книгами, и связанные страницы на диаграмме классов (рисунок 2.10). Также представлено сохранение и загрузка достижений.

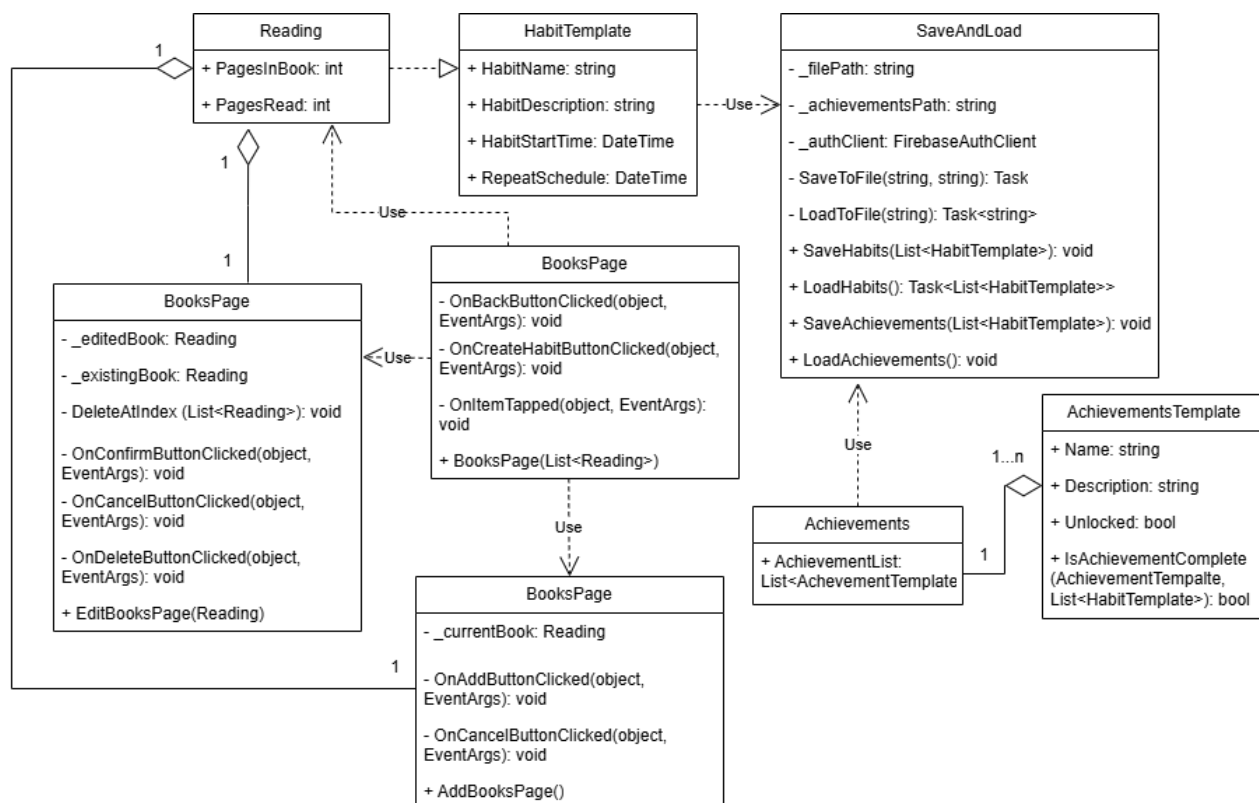


Рисунок 2.10 – Диаграмма привычек для книг

Остальные страницы привычек сделаны аналогично, меняется только класс используемых привычек. В классе сохранения из-за полиморфизма все привычки сохраняются в одном списке, сохраняя свой изначальный класс, а не класс родителя. Достижения сохраняются вместе с привычками.

Страница привычки разделена на три отдельных: список с привычками, страница добавления и страница изменения. Классы страниц содержат события для обработки действий пользователя, визуальная составляющая находит отдельно, в файле формата xaml.

Теперь рассмотрим диаграмму классов для дневника настроения (рисунок 2.11).

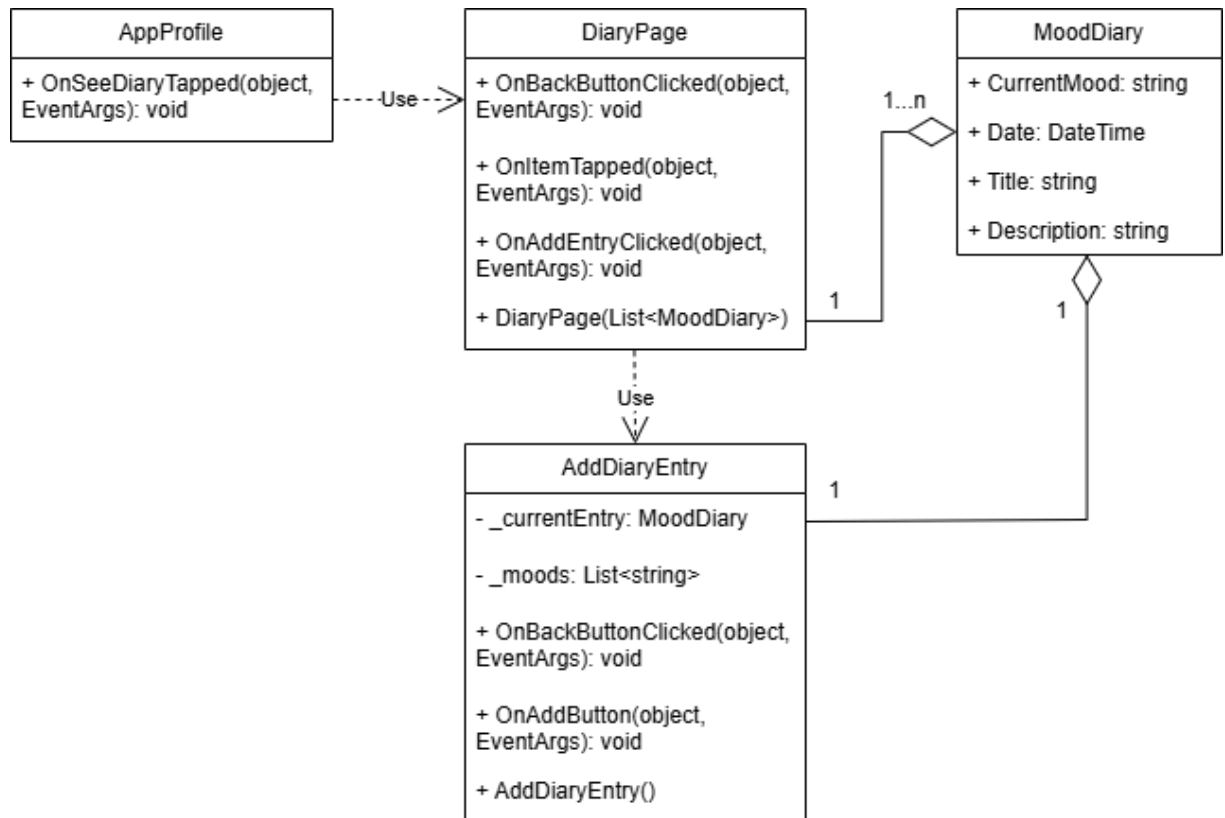


Рисунок 2.11 – Диаграмма дневника настроения

Переход на дневник настроения проводится через профиль пользователя. В дневнике можно только просматривать записи через страницу дневника настроения и добавлять записи на странице добавления. Для дневника используется класс **MoodDiary**, которых содержит модель для одной записи в дневнике, а список всех записей показывается на странице **DiaryPage**.

3 ТЕСТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА

3.1 Система создания и редактирования привычек

Программа была запущена на операционной системе Windows. На рисунке 3.1 изображено главное окно программы, на нем находится окно с прогрессией, кнопки для меню привычек, а также нижнее меню, которое открывает другие страницы.

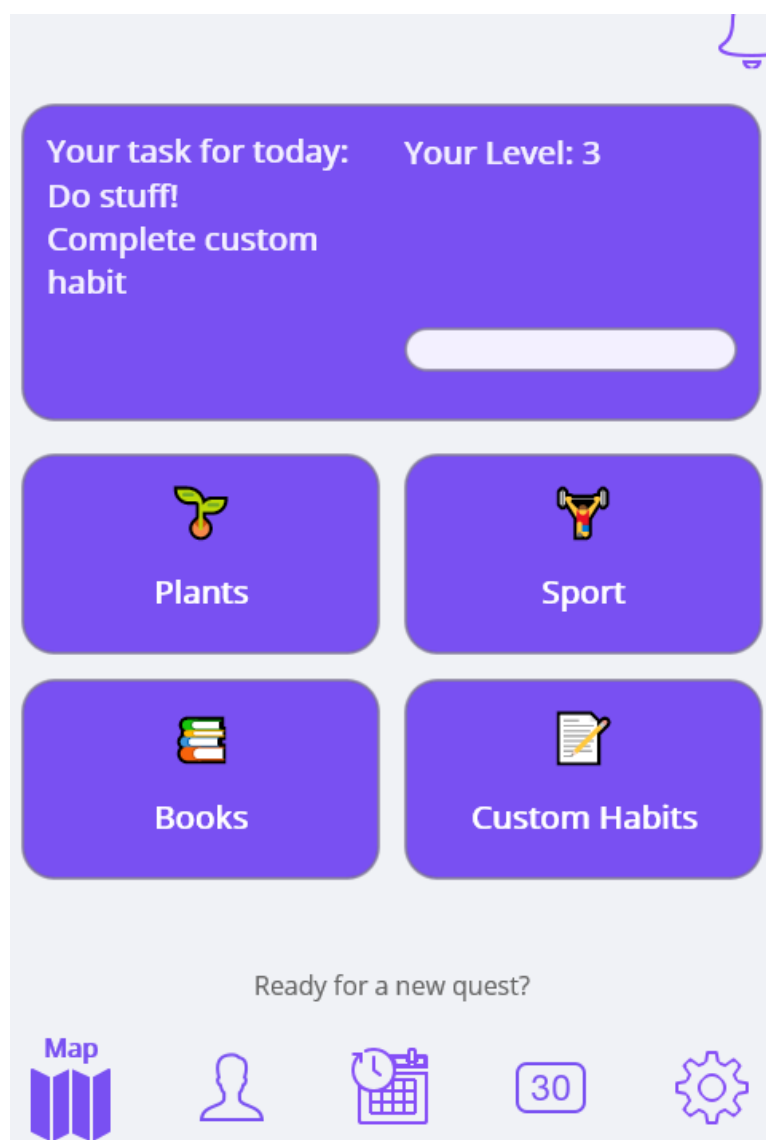


Рисунок 3.1 – Главное окно приложения

Страницы всех привычек одинаковы, различается только данные, которые необходимо заполнить: для растений и своих привычек окна

одинаковые, для тренировок нужно заполнить вес, который нужно достичь, а для книг количество страниц в книге.

Попробуем создать свою привычку, процесс создания изображен на рисунках 3.2 и 3.3.

The screenshot shows a mobile app interface titled "Add Habit". At the top is a purple header bar with a white back arrow icon on the left and the title "Add Habit" in white text. Below the header, the form is divided into several sections with light gray backgrounds. The first section is labeled "Habit Name:" and contains the text "Clean apartment". The second section is labeled "Description:" and contains the text "Clean whole flat". The third section is labeled "Additional Tasks:" and contains two items: "Mop the floors" and "Wash dishes", each on a separate line. Below these sections is a large purple button with the text "Add Task" in white. Underneath the button is a section labeled "Repeat Schedule:" which shows the date "12.06.2025" followed by a small calendar icon. At the bottom of the form is a large purple button with the text "Add" in white.

Рисунок 3.2 – Создание своей привычки

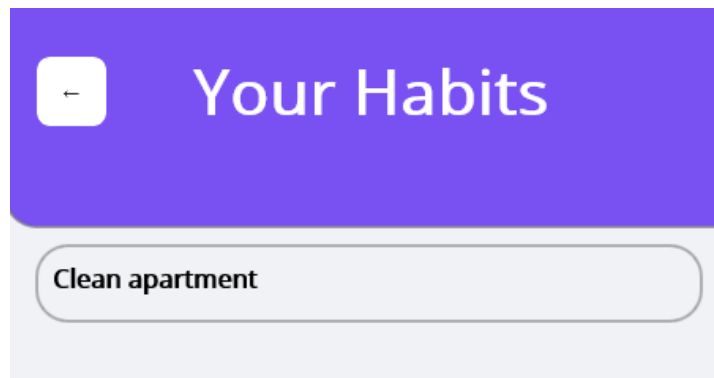


Рисунок 3.3 – Отображение привычки в списке

После создания привычки теперь мы можем её изменить и пометить как выполненную (рисунки 3.4 и 3.5).

View your habit or edit it!

Habit Name:

Clean apartment

Description:

Clean whole flat

Additional Tasks:

Mop the floors

Wash dishes

Repeat Schedule:

12.06.2025

Is habit complete?

☐

Confirm **Delete**

Рисунок 3.3 – Изменение привычки

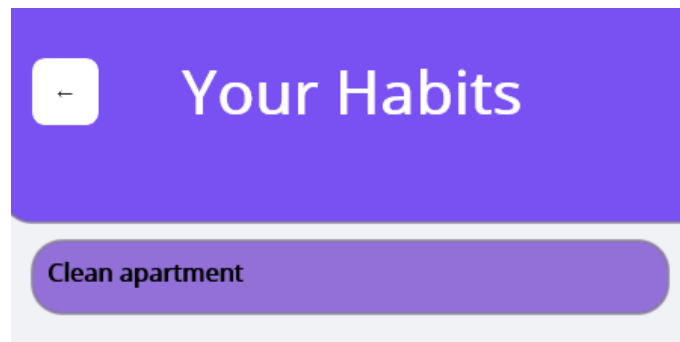


Рисунок 3.4 – Измененная привычка отображается иначе

В привычках, связанных с растениями, выполнением считается полив растения раз в два дня. Для тренировок нужно достичь вес, который был выбран при создании. Для книг нужно прочитать все страницы.

3.2 Календарь и дневник настроения

Посмотрим календарь, созданные привычки отображаются точками в календаре, при выборе определенной даты указывается список привычек, которые необходимо выполнить (рисунок 3.5).

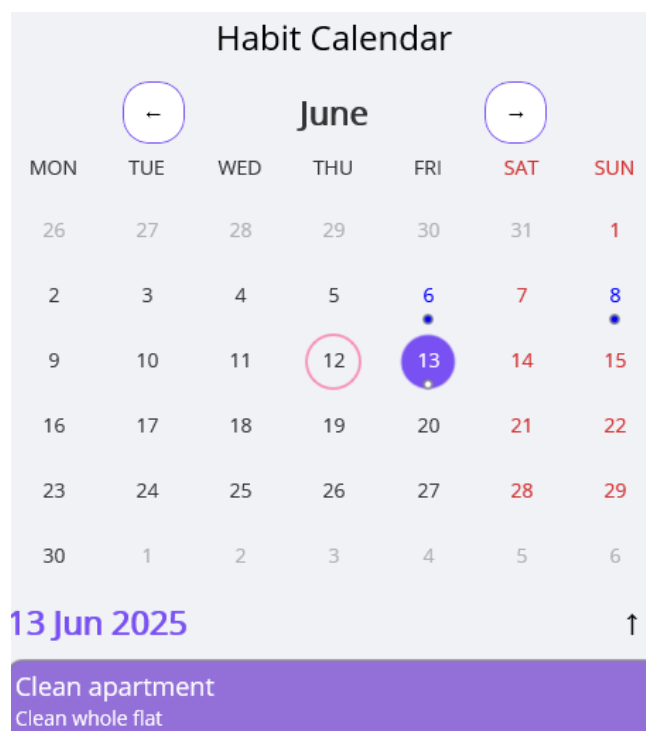


Рисунок 3.5 – Календарь привычек

Для программного комплекса был создан дневник настроения, чтобы человек мог отслеживать какой день был более продуктивным, а какой был менее продуктивным. Дневник настроения находится в профиле (рисунок 3.6), в дневник можно только добавлять и просматривать записи (рисунки 3.7 и 3.8).

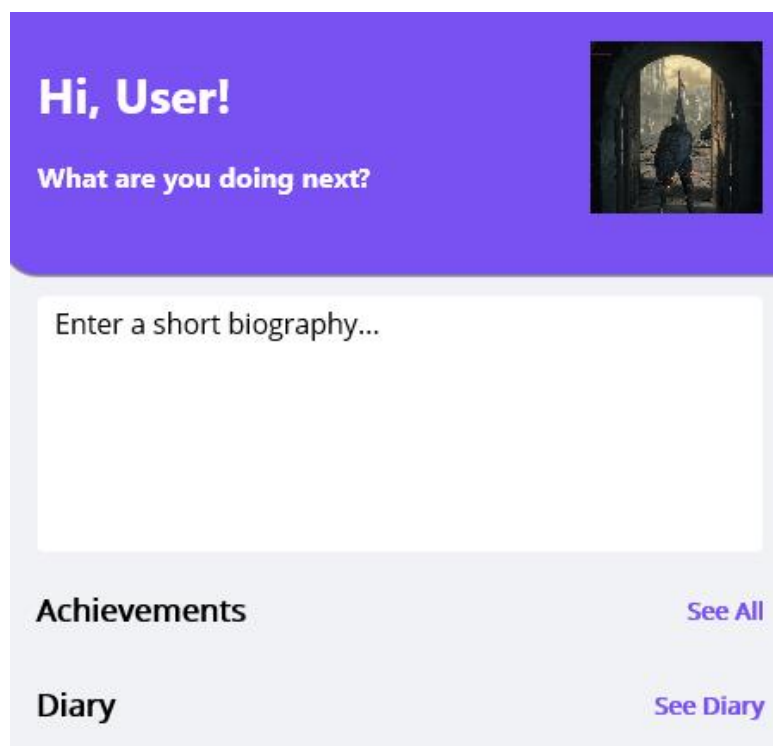


Рисунок 3.6 – Профиль пользователя

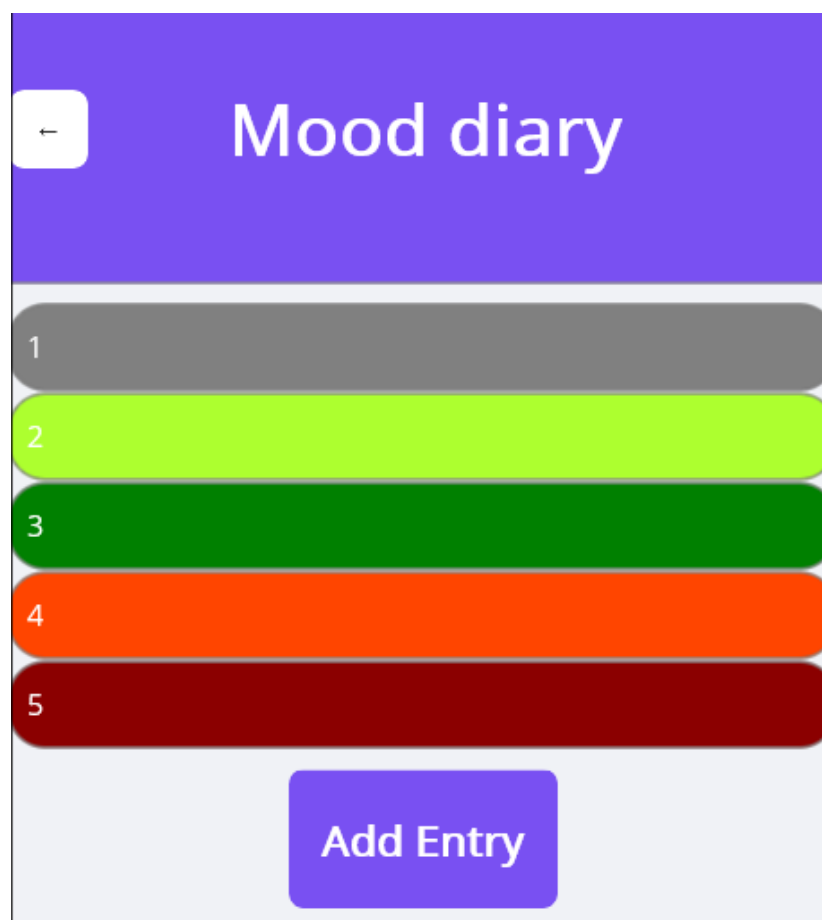


Рисунок 3.7 – Дневник настроения

1
1
Neutral
02.06.2025 16:09:16

Рисунок 3.8 – Запись в дневнике

В дневнике цветом помечается выбранное настроение в определенный день (можно выбрать отличное настроение, хорошее, нейтральное, плохое и ужасное). В записе указано название, описание, какое было настроение и дата создания записи.

3.3 Отслеживание прогресса и система достижений

На главном окне присутствует окно с прогрессией (рисунок 3.9), на нем указан текущий уровень пользователя, количество опыта до следующего уровня и какое задание надо выполнить чтобы получить опыт.

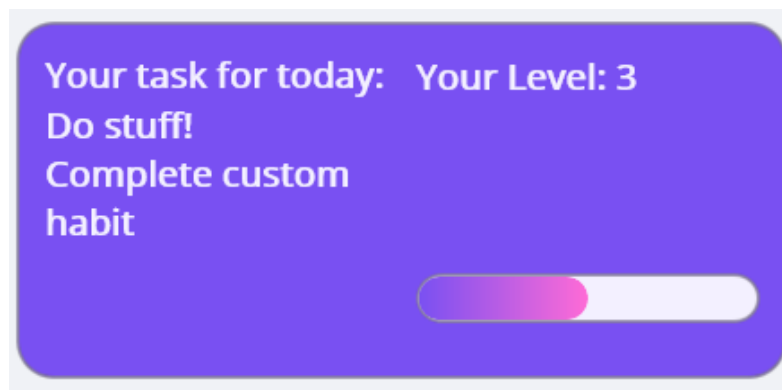


Рисунок 3.9 – Окно прогрессии

Если выполнить задание, то пользователь сможет получить новый уровень и появится окно, которое говорит о повышении уровня (рисунок 3.10).

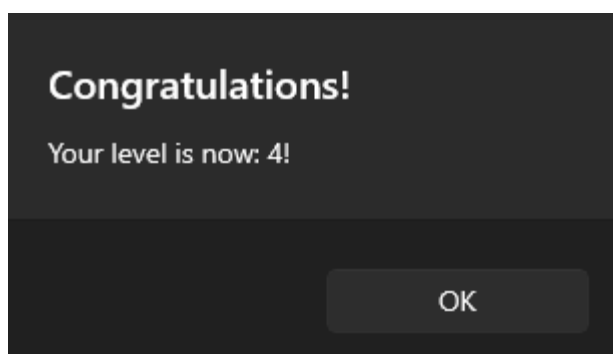


Рисунок 3.10 – Уведомление о повышении уровня

Также на главном окне расположено окно со статистикой (рисунок 3.11) в котором указано сколько всего выполнено привычек в каждой из категорий и сколько максимально дней подряд была совершена привычка. Ниже этого окна находится диаграмма, которая показывает эту информацию в ином виде (рисунок 3.12).



Рисунок 3.11 – Окно статистики

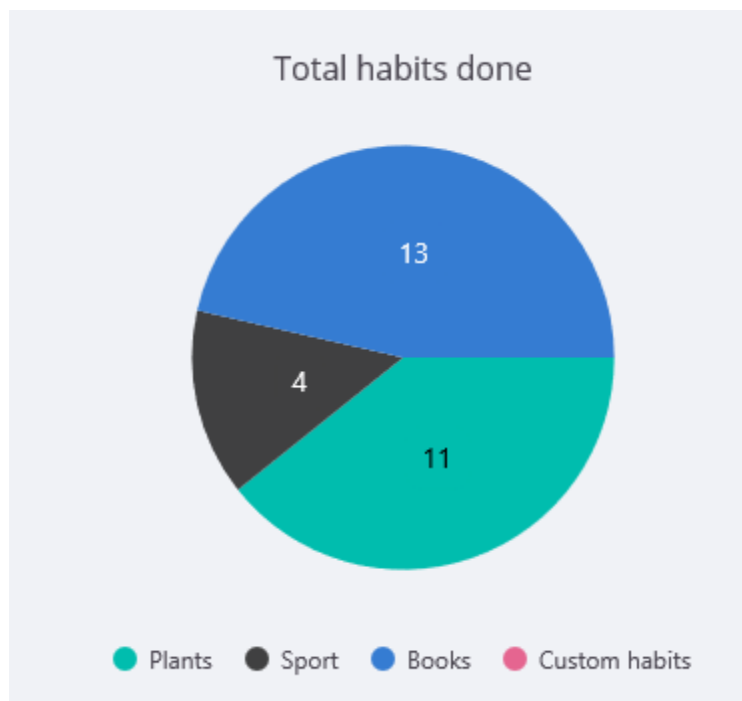


Рисунок 3.12 – Диаграмма выполненных привычек

Окно с достижениями (рисунок 3.13) находится в профиле пользователя, список достижений содержит всего 13 достижений. При выполнении условий появляется уведомление (рисунок 3.14) в котором указано открытое достижение и меняется иконка в списке достижений.

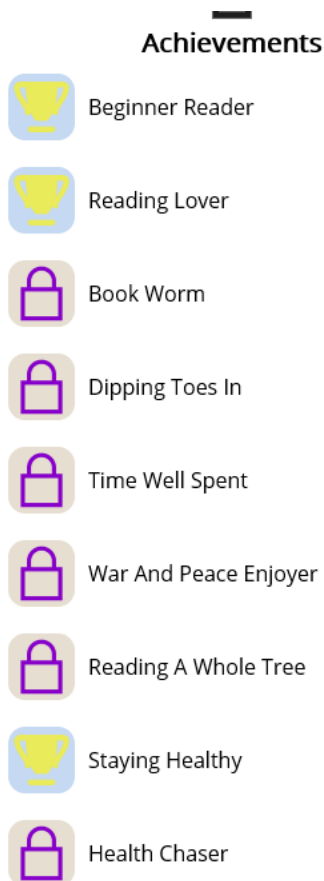


Рисунок 3.13 – Список достижений

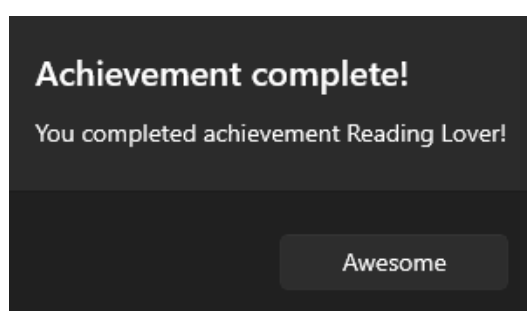


Рисунок 3.14 – Уведомление при выполнении условий достижения

Таким образом, были рассмотрены все основные функциональные возможности программного комплекса для создания полезных привычек и повышения личной эффективности.

Заключение

В ходе данной работы было разработано кроссплатформенное приложение «Habitualize», которое поможет пользователю с планированием и повышением личной эффективности.

Были выполнены все поставленные задачи:

1) разработан интерфейс и логика для создания и редактирования привычек - пользователь может создать привычки четырех типов, каждая из которых по-своему проверяет выполнена она или нет. Также интерфейс создан адаптивным, чтобы на любой платформе, не зависимо от размеров экрана, приложение выглядело одинаково;

2) создан календарь для отслеживания привычек по дням – календарь поможет пользователю визуализировать какие привычки нужно выполнить в определенный день, это поможет с планированием и как следствие увеличит работоспособность пользователя;

3) разработана система достижений и отслеживания прогресса достижений – система достижений создана с помощью принципов геймификации, которые увеличивают степень вовлеченности пользователя и стимулирует пользователя заходить каждый день в приложение, что поможет привить полезные привычки;

4) создан интерфейс для просмотра статистики по созданными привычкам – статистика позволит пользователю видеть свой прогресс, что позволит пользователю стремиться для улучшения своих результатов;

5) установлено подключение к облачной базе данных Firebase для сохранения и загрузки пользовательских данных.

Подключение позволяет синхронизировать данные между несколькими устройствами одновременно, что увеличивает доступность приложения на различных платформах.

Данное приложение можно будет усовершенствовать, например добавить награды за определенные вехи при получении уровня, можно

добавить награды за открытие достижений. Всё это сможет позволить повысить вовлеченность пользователя при использовании приложения.

Список использованных источников

- 1 Singularity (To Do List, Tasks) | SingularityApp [Электронный ресурс]. URL: <https://singularity-app.ru/>
- 2 Трекер привычек: Tick It | Tick It [Электронный ресурс]. URL: <https://apps.apple.com/us/app/%D1%82%D1%80%D0%B5%D0%BA%D0%B5%D1%80-%D0%BF%D1%80%D0%B8%D0%B2%D1%8B%D1%87%D0%B5%D0%BA-tick-it/id1550577355?l=ru>
- 3 Everyday трекер привычек | Everyday [Электронный ресурс]. URL: <https://play.google.com/store/apps/details?id=com.everydaycheckmobile&hl=ru>
- 4 Статистика выгорания в России | Forbes [Электронный ресурс]. URL: <https://www.forbes.ru/forbeslife/511862-43-rossian-ispytyvali-professional-noe-vygoranie> (дата обращения 20.05.2025 г.)
- 5 Документация по Visual Studio | Microsoft Learn [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/visualstudio/windows/?view=vs-2022> (дата обращения: 22.05.2025 г.)
- 6 Документация по .NET MAUI | Microsoft Learn [Электронный ресурс]. URL: <https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui?view=net-maui-8.0> (дата обращения: 20.05.2025 г.)
- 7 Руководство по программированию на .NET MAUI и C# | Metanit [Электронный ресурс]. URL: <https://metanit.com/sharp/maui/> (дата обращения 20.05.2025 г.)
- 8 Документация по Firebase Authentication | Firebase [Электронный ресурс]. URL: <https://firebase.google.com/docs/auth> (дата обращения: 20.05.2025 г.)
- 9 Документация по Firebase Realtime Database | Firebase [Электронный ресурс]. URL: <https://firebase.google.com/docs/database> (дата обращения: 20.05.2025 г.)
- 10 Подключение Firebase к приложению | FlutterFlow Documentation

[Электронный ресурс]. URL: <https://docs.flutterflow.io/integrations/firebase/connect-to-firebase/> (дата обращения: 20.05.2025 г.)

11 Настройка правил безопасности Firebase Realtime Database | Firebase [Электронный ресурс]. URL: <https://firebase.google.com/docs/database/security> (дата обращения: 20.05.2025 г.)

12 Коцубинский В. П., Изюмов А. А., Хабибуллина Н. Ю. Производственная практика бакалавра направления подготовки 27.03.04 Управление в технических системах: методические указания по выполнению производственной практики, в том числе практики по получению профессиональных умений и опыта профессиональной деятельности, технологической и преддипломной практики. Томск: каф. КСУП, ТУСУР, 2016. 113с. [Электронный ресурс]. URL: <https://edu.study.tusur.ru/publications/6324/download> (дата обращения: 20.05.2025 г.)

13 ОС ТУСУР 01-2021. Работы студенческие по направлениям подготовки и специальностям технического профиля. Общие требования и правила оформления. Томск: Томский гос. ун-т систем упр. и радиоэлектроники, 2021. 52 с. [Электронный ресурс]. URL: https://regulations.tusur.ru/system/document_files/files/000/000/094/original/OS_TUSUR_01-2021_%28soglasovano%29.pdf?1735368202 (дата обращения: 20.05.2025 г.)

14 Преддипломная практика: Рабочая программа учебной дисциплины «Преддипломная практика»/ Черкашин М. В., Горяинов А. Е. 2023 [Электронный ресурс]. URL: https://edu.tusur.ru/work_programs/76560/download (дата обращения: 20.05.2025 г.)